

UNIBRA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
SEGURANÇA DA INFORMAÇÃO
LISTA 01

Parte I: Questões Discursivas

QUESTÃO DISCURSIVA 1

TEXTO I A arquitetura Cliente/Servidor é um modelo computacional que divide tarefas e cargas de trabalho entre os fornecedores de um recurso ou serviço, chamados de servidores, e os requerentes dos serviços, chamados de clientes. Na web moderna, essa comunicação ocorre majoritariamente via protocolo HTTP.

TEXTO II Uma empresa decidiu modernizar seu sistema de gestão de estoque, que antes era uma aplicação *Desktop* monolítica rodando na máquina de cada funcionário, para uma aplicação Web. O novo sistema terá uma interface em React (Cliente) rodando no navegador e uma API em Java (Servidor) rodando na nuvem.

A partir da leitura dos textos e considerando os conceitos de redes e desenvolvimento web, atenda ao que se pede:

- a)** Diferencie as responsabilidades da camada Cliente e da camada Servidor neste novo cenário do sistema de estoque. Quais tipos de processamento devem ser feitos em cada ponta?
- b)** O protocolo HTTP é, por definição, *stateless* (sem estado). Explique o que isso significa na comunicação entre o navegador do funcionário e a API em Java, e cite uma estratégia técnica comum (ex: tokens ou sessões) para que o servidor consiga "lembrar" que o usuário já fez o login nas requisições subsequentes.

QUESTÃO DISCURSIVA 2

TEXTO ÚNICO A linguagem Java é fortemente tipada e orientada a objetos, sendo amplamente utilizada na construção de servidores robustos. Em um sistema bancário simples, um desenvolvedor criou a classe *ContaCorrente* com os atributos *numeroDaConta* e *saldo*. Ele também precisa implementar um

método `transferir(ContaCorrente destino, double valor)`. As regras de negócio exigem que uma transferência não possa ser concluída se o saldo for insuficiente, e o sistema deve alertar a camada superior sobre esse problema.

Com base nos princípios de Programação Orientada a Objetos em Java, faça o que se pede:

- a) Explique como o conceito de Encapsulamento deve ser aplicado na classe `ContaCorrente` para proteger o atributo `saldo`. Como os modificadores de acesso e os métodos *Getters/Setters* atuam nesse controle?
- b) Descreva como o mecanismo de Tratamento de Exceções do Java (blocos `try`, `catch`, `finally` ou a palavra-chave `throw`) deve ser utilizado no método `transferir` para impedir a transação em caso de saldo insuficiente e notificar o sistema de forma limpa, sem quebrar a execução do servidor.

QUESTÃO DISCURSIVA 3

TEXTO I O padrão de arquitetura MVC (Model-View-Controller) separa uma aplicação em três componentes lógicos principais. Essa separação ajuda a gerenciar a complexidade de aplicações grandes, permitindo que desenvolvedores de *frontend* e *backend* trabalhem de forma mais independente, além de facilitar a manutenção e a testabilidade do código.

TEXTO II Um aluno de ADS está desenvolvendo um portal de notícias universitário. Ao digitar a URL de uma notícia no navegador e apertar "Enter", a página é carregada com o título, o texto e os comentários do banco de dados.

Considerando o fluxo de funcionamento do padrão MVC em uma aplicação web, responda:

- a) Defina claramente a responsabilidade de cada uma das três camadas (Model, View e Controller) no contexto deste portal de notícias.
- b) Descreva o passo a passo do fluxo da requisição HTTP gerada pelo usuário ao acessar a URL da notícia, passando por todas as camadas do padrão MVC, desde a chegada ao servidor até a devolução da página HTML pronta para o navegador.

QUESTÃO DISCURSIVA 4

TEXTO I O ecossistema Spring, especialmente com o advento do Spring Boot, revolucionou a forma como aplicações Java corporativas são construídas. Ele abstrai grande parte da configuração de infraestrutura ("boilerplate"), permitindo que o desenvolvedor foque na regra de negócio. Dois dos conceitos mais poderosos implementados pelo Spring são a Inversão de Controle (IoC) e a Injeção de Dependências (DI).

No contexto de desenvolvimento de um servidor backend com Spring Boot, elabore um texto abordando os seguintes pontos:

- a) Explique o que é a Inversão de Controle (IoC) e como o *Container* do Spring assume o ciclo de vida dos objetos (os *Beans*).
- b) Explique o conceito de Injeção de Dependências (DI). Utilize como exemplo uma classe *ClienteController* que precisa acessar métodos de uma classe *ClienteService*, demonstrando conceitualmente (ou com anotações como `@Autowired`) como o Spring resolve essa dependência sem que o desenvolvedor precise usar a palavra reservada `new`.

QUESTÃO DISCURSIVA 5

TEXTO ÚNICO A criação de APIs RESTful é uma das aplicações mais comuns da programação Cliente/Servidor atualmente. Uma equipe precisa expor os dados de um catálogo de produtos armazenados em um banco de dados relacional para que um aplicativo de celular (o cliente) possa exibí-los. Para isso, utilizarão o framework Spring Boot para construir os *endpoints* da API.

Tendo em vista a construção de APIs modernas em Java, atenda ao que se pede:

- a) Na comunicação entre o aplicativo móvel e a API feita em Spring Boot, qual é o formato de representação de dados mais utilizado atualmente (substituto moderno do XML) para enviar e receber os dados dos produtos? Descreva brevemente as características desse formato que o tornam ideal para a web.
- b) No desenvolvimento do Controlador (*Controller*) dessa API em Spring Boot, explique qual é a função das anotações `@RestController` e `@GetMapping` (ou

@RequestMapping). Como elas transformam uma classe Java comum em um ponto de entrada acessível via requisição HTTP?

Parte II

Texto Único A arquitetura web moderna baseia-se fortemente no modelo Cliente/Servidor utilizando o protocolo HTTP (Hypertext Transfer Protocol). O crescimento no uso de dispositivos móveis e frameworks de *frontend* (como React ou Angular) consolidou a separação estrita entre a camada de apresentação e a camada de processamento de regras de negócio.

Sobre a comunicação Cliente/Servidor via HTTP em aplicações web, avalie as afirmações a seguir:

- I. O protocolo HTTP é classificado como *stateless* (sem estado), o que significa que o servidor não guarda naturalmente o contexto das requisições anteriores de um mesmo cliente.
- II. Para contornar a natureza *stateless* do HTTP em sistemas que exigem autenticação, os servidores utilizam mecanismos como *Cookies*, Sessões em memória ou Tokens (ex: JWT) trafegados nos cabeçalhos (*headers*) da requisição.
- III. Na arquitetura Cliente/Servidor tradicional, o cliente é responsável por gerenciar a conexão com o banco de dados relacional e aplicar as regras de negócio antes de renderizar a tela.

É correto o que se afirma em:

- A) I, apenas.
- B) III, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

QUESTÃO 2

Texto I O padrão arquitetural MVC divide uma aplicação interativa em três componentes principais. O objetivo dessa separação é desacoplar a lógica de acesso a dados, a lógica de negócio e a interface com o usuário.

A respeito das responsabilidades no padrão MVC, avalie as asserções a seguir e a relação proposta entre elas.

- I. O componente **Controller** atua como um intermediário que recebe as requisições HTTP do usuário, processa os dados de entrada e decide qual resposta deve ser devolvida. PORQUE
- II. O **Controller** é o componente responsável por armazenar o estado da aplicação, validar as regras de negócio complexas e executar as instruções SQL diretamente no banco de dados.

A respeito dessas asserções, assinale a opção correta:

- A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E) As asserções I e II são proposições falsas.

QUESTÃO 3 No desenvolvimento da camada de *Model* de um sistema acadêmico em Java, um programador criou a classe *Aluno* contendo os atributos *nome*, *matricula* e *mediaFinal*. Para garantir a integridade dos dados em memória, ele aplicou o conceito de Encapsulamento.

Considerando as boas práticas de programação em Java, o Encapsulamento correto para essa classe exige que:

- A) Os atributos sejam declarados como *public* para facilitar o acesso pelo *Controller*, evitando a criação de métodos auxiliares.
- B) A classe *Aluno* herde da classe *Controller*, utilizando a palavra reservada *extends*.
- C) Os atributos sejam declarados como *private*, e o acesso a eles seja feito exclusivamente por meio de métodos públicos de leitura e escrita (*Getters* e *Setters*), onde validações podem ser aplicadas.
- D) O atributo *mediaFinal* seja declarado como *static*, garantindo que todos os objetos instanciados da classe *Aluno* compartilhem a mesma nota.

E) A classe implemente a interface `Serializable` e declare todos os seus métodos como `protected`.

QUESTÃO 4 O Spring Boot simplifica o desenvolvimento de aplicações Java, especialmente na criação de APIs RESTful. Ele utiliza anotações (metadados) para configurar comportamentos sem a necessidade de extensos arquivos XML.

Considere um desenvolvedor que precisa criar um *endpoint* que retorne a lista de produtos de um catálogo em formato JSON ao receber uma requisição HTTP do tipo GET.

Quais anotações do ecossistema Spring Boot o desenvolvedor deve utilizar na classe e no método, respectivamente, para alcançar esse objetivo?

- A) `@Service` na classe e `@PostMapping` no método.
- B) `@RestController` na classe e `@GetMapping` no método.
- C) `@Repository` na classe e `@PutMapping` no método.
- D) `@Component` na classe e `@RequestMapping` no método.
- E) `@Controller` na classe e `@DeleteMapping` no método.

QUESTÃO 5 A transferência de estado representacional (REST) é um estilo de arquitetura que define um conjunto de restrições para a criação de *web services*. Em uma API RESTful madura, os métodos do protocolo HTTP devem ser utilizados de forma semântica, respeitando suas definições padronizadas. Correlacione os métodos HTTP a seguir com suas respectivas ações padrão em um recurso "Cliente":

1. GET
2. POST
3. PUT
4. DELETE

- () Atualiza/Substitui os dados de um cliente existente no servidor.
- () Remove o registro de um cliente do servidor.
- () Recupera os dados de um ou mais clientes do servidor sem alterá-los.
- () Cria um novo registro de cliente no servidor.

A sequência correta, de cima para baixo, é:

- A) 3, 4, 1, 2
- B) 2, 4, 1, 3
- C) 3, 1, 4, 2
- D) 1, 2, 3, 4
- E) 4, 3, 2, 1

QUESTÃO 6

Texto I

Historicamente, no desenvolvimento Java tradicional, o programador era responsável por instanciar explicitamente os objetos necessários utilizando a palavra-chave `new` (ex: `PedidoService service = new PedidoService();`), o que gerava alto acoplamento entre as classes.

Texto II

O framework Spring resolve esse problema implementando o conceito de Inversão de Controle (IoC), no qual um contêiner central passa a gerenciar o ciclo de vida dos objetos (chamados de *Beans*).

A funcionalidade do Spring que permite que um objeto receba suas dependências automaticamente do contêiner, sem precisar instanciá-las manualmente, é conhecida como:

- A) Polimorfismo.
- B) Injeção de Dependências (Dependency Injection).
- C) Mapeamento Objeto-Relacional (ORM).
- D) Herança Múltipla.
- E) Sobrecarga de Métodos.

QUESTÃO 7

Durante o processamento de uma requisição em uma API construída em Java, uma classe de serviço tenta realizar a conversão de uma String (recebida do cliente web) para um número Integer para realizar um cálculo matemático. No entanto, o cliente enviou letras em vez de números, gerando uma exceção do tipo `NumberFormatException`.

Para que o servidor não interrompa sua execução de forma abrupta (crash) e possa devolver uma mensagem de erro amigável ao cliente web (ex: "Formato inválido"), o desenvolvedor deve utilizar a estrutura:

- A) if - else if - else no mapeamento da rota.
- B) switch - case envolvendo a conversão do dado.
- C) while - do para tentar converter repetidamente até o cliente acertar.
- D) try - catch envolvendo o bloco de código de conversão.
- E) extends Exception na assinatura da classe de serviço.

QUESTÃO 8 No desenvolvimento de aplicações corporativas com Spring Boot, convencionou-se adotar uma arquitetura em camadas para organizar o código. As camadas mais comuns são Controller, Service e Repository.

Avalie as afirmações sobre as responsabilidades dessas camadas:

- I. A camada **Controller** é o ponto de entrada da API, responsável por expor as rotas HTTP e receber os *requests* do cliente.
- II. A camada **Service** é onde as regras de negócio complexas, cálculos e orquestrações devem ser programadas.
- III. A camada **Repository** (frequentemente utilizando o Spring Data JPA) tem o papel exclusivo de renderizar as páginas HTML para o navegador do usuário final.

É correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

QUESTÃO 9 Em um projeto de disciplina, foi solicitado o desenvolvimento de uma aplicação monolítica em Spring Boot que implemente o MVC clássico utilizando *Thymeleaf* como mecanismo de *template* (para as *Views*).

Nesse modelo específico, a resposta (Response) devolvida pelo servidor web ao navegador do cliente (após o Controller processar o Model e passá-lo para a View) será em formato:

- A) Um documento HTML renderizado contendo os dados dinâmicos.
- B) Uma estrutura de dados binária compilada em *Bytecode* do Java.
- C) Exclusivamente um objeto JSON com os atributos do banco de dados.
- D) Um script SQL contendo os comandos de SELECT executados pelo *Model*.
- E) Um arquivo .JAR para ser executado localmente na máquina do cliente.

QUESTÃO 10 Sistemas modernos frequentemente combinam diferentes técnicas criptográficas para aproveitar as melhores características de cada uma. O protocolo TLS, base da navegação segura na internet, opera com um conceito de "Criptografia Híbrida".

Sobre o funcionamento desse modelo no TLS, é correto afirmar que:

- A) Utiliza a criptografia simétrica para trocar a chave pública e, em seguida, usa a criptografia assimétrica para transferir os dados do site de forma mais rápida.
- B) Utiliza a criptografia assimétrica inicialmente para autenticar o servidor e realizar a troca segura de uma chave de sessão temporária, passando a utilizar a criptografia simétrica para encriptar o tráfego de dados subsequente.
- C) Elimina a necessidade de Autoridades Certificadoras (ACs), pois tanto o cliente quanto o servidor geram chaves simétricas baseadas em funções de Hash unidirecionais.
- D) Aplica assinaturas digitais em todos os pacotes de rede transferidos utilizando o algoritmo AES, garantindo a confidencialidade contínua.
- E) Utiliza apenas chaves públicas de 2048 bits para criptografar todo o tráfego TCP/IP, descartando o uso de chaves privadas devido à alta latência.